

UNIT-2

Models of ANN

Soft Computing(TCS821)

by

Dr. Manoj Singh Adhikari

Supervised learning

2

- In this learning, every input pattern that is used to train the network is associated with an output pattern.
- This is called training set of data. Thus, in this form of learning, the input-output relationship of the training scenarios are available.
- Here, the output of a network is compared with the corresponding target value and the error is determined.
- It is then feed back to the network for updating the same. This results in an improvement.
- This type of training is called learning with the help of teacher.

Supervised learning

3

Supervised learning is a machine learning paradigm that trains algorithms using labeled datasets to map input data to specific, known outputs. By analyzing these labeled input-output pairs, the model learns underlying patterns to predict outcomes on new, unseen data, commonly used for classification and regression tasks.

Key aspects of Supervised learning

4

Labeled Data: The model is trained on data where the "ground truth" (correct answer) is already known, acting as a teacher for the algorithm.

Goal: The primary objective is to generalize from the training data to accurately predict the output for new, unseen data

Types of Problems: Classification: Categorizes data into specific, distinct classes (e.g., spam detection, image recognition).

Regression: Predicts continuous, numerical values (e.g., house prices, sales forecasting).

Common Algorithms: Linear regression, logistic regression, support vector machines (SVM), decision trees, and random forests.

Applications: Fraud detection, spam filtering, image classification, and risk assessment.

Unsupervised learning

5

- If the **target output is not available**, then the **error in prediction can not be determined** and in such a situation, the system learns of **its own** by **discovering and adapting to structural features in the input patterns**.
- This type of training is called **learning without a teacher**.

Unsupervised learning

6

- Unsupervised learning is a type of machine learning that analyzes and clusters unlabeled datasets to discover hidden patterns, structures, or data groupings without human intervention. Unlike supervised learning, it operates without predefined labels or target outcomes, making it ideal for exploratory data analysis, customer segmentation, anomaly detection, and data compression.

Key aspects of Unsupervised learning

7

Clustering: Algorithms (e.g., K-Means, Hierarchical) group similar data points together based on inherent characteristics.

Dimensionality Reduction: Techniques (e.g., PCA, Autoencoders) reduce the number of variables in a dataset while retaining essential information, simplifying data for further analysis.

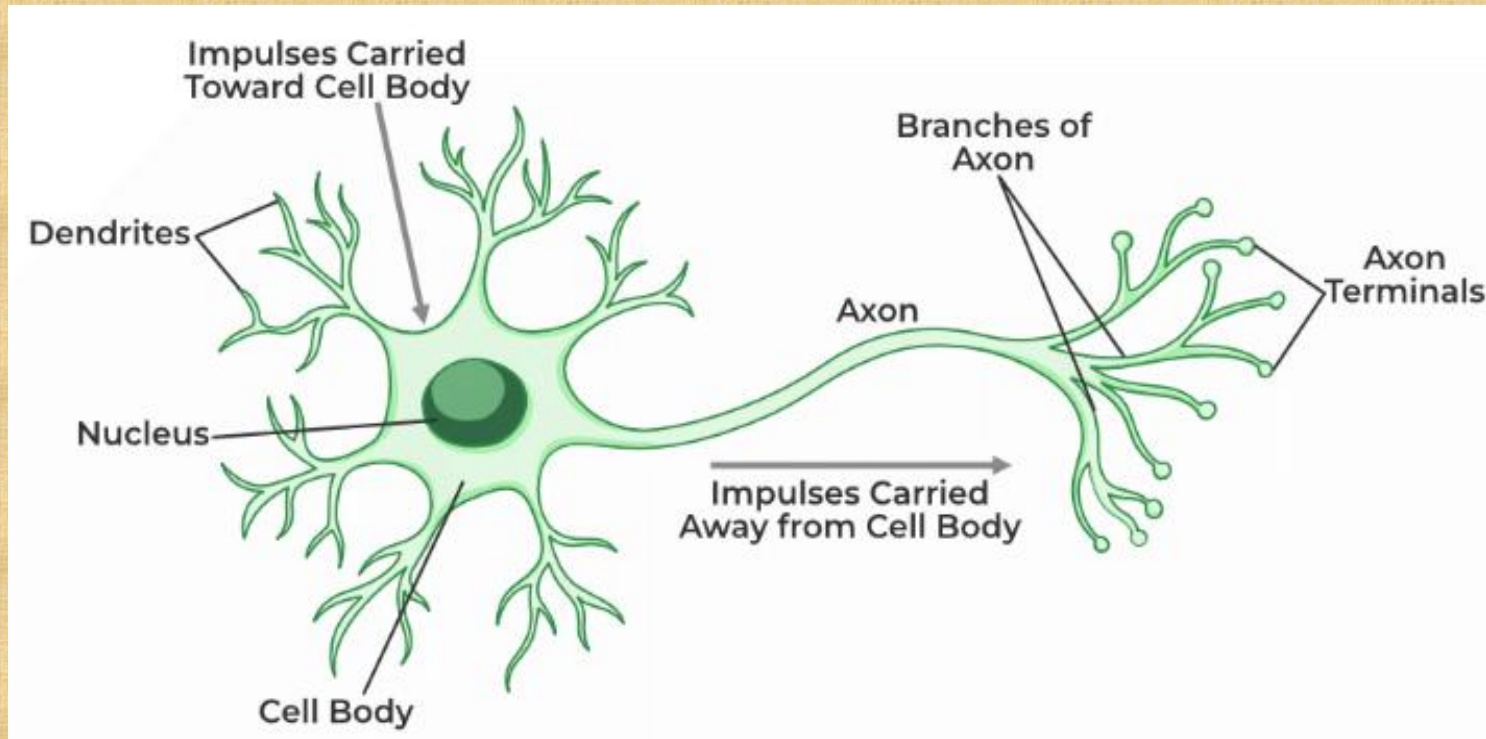
Association Rules: Identifies relationships and rules that describe large portions of data, such as items frequently purchased together.

Anomaly Detection: Detects rare or unusual items, events, or observations that deviate significantly from the majority of the data.

Single layer perception (SLP)

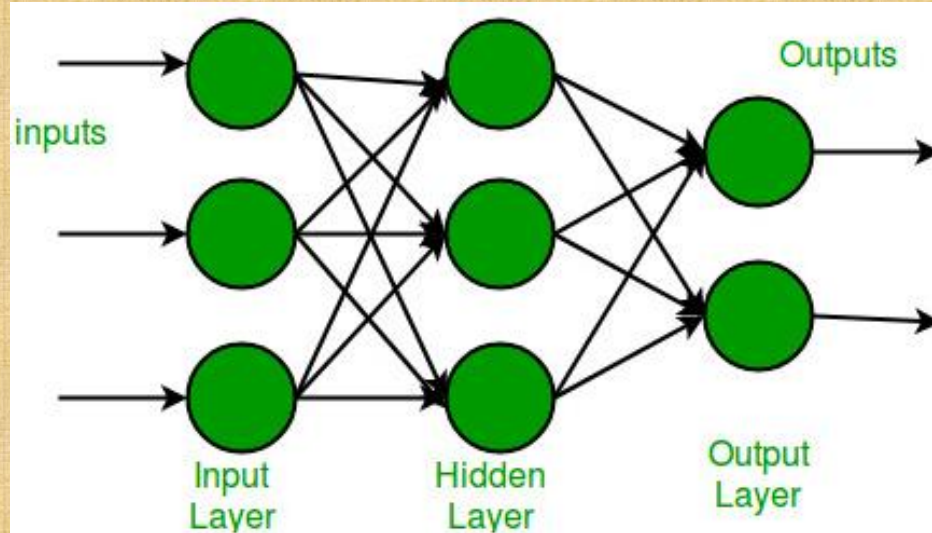
Single Layer Perceptron is **inspired by biological neurons** and their ability to process information. To understand the SLP we first need to break down the workings of a single artificial neuron which is the fundamental building block of neural networks. An **artificial neuron** is a simplified computational model that mimics the behavior of a biological neuron. It takes inputs, processes them and produces an output. Here's how it works step by step:

- Receive signal from outside.
- Process the signal and decide whether we need to send information or not.
- Communicate the signal to the target cell, which can be another neuron or gland.



Structure of a biological neuron

Similarly, neural networks also function in a similar manner.



Neural Network in Machine Learning

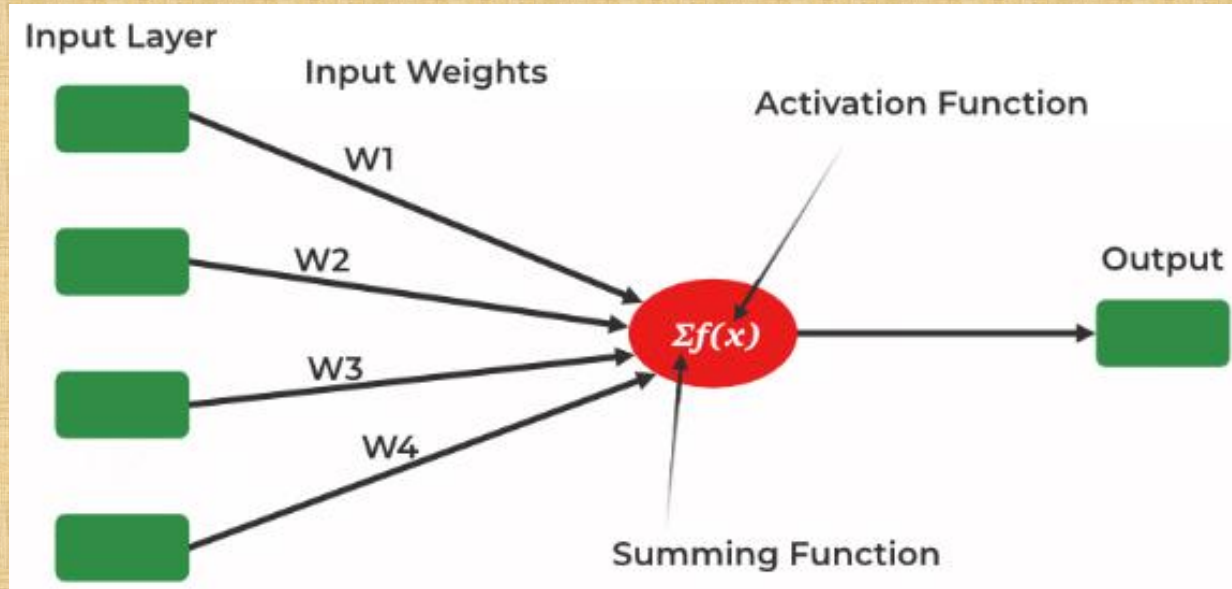
Single Layer Perceptron

It is one of the oldest and first introduced neural networks. It was proposed by **Frank Rosenblatt** in **1958**. Perceptron is also known as an artificial neural network. Perceptron is mainly used to compute the [logical gate](#) like **AND, OR and NOR** which has binary input and binary output.

The main functionality of the perceptron is:-

- Takes input from the input layer
- Weight them up and sum it up.
- Pass the sum to the nonlinear function to produce the output.

Single Layer Perceptron



Single-layer neural network

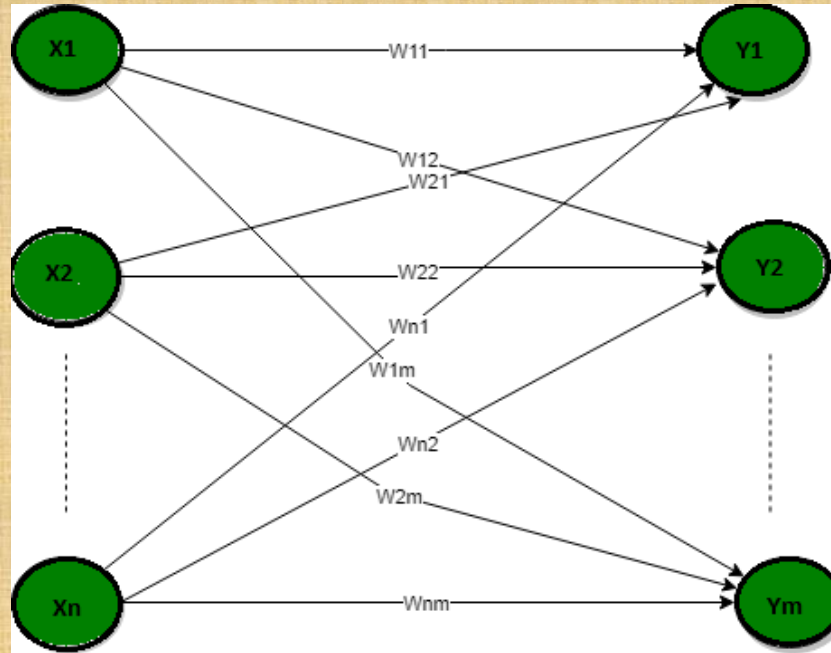
Here activation functions can be anything like **sigmoid**, **tanh**, **relu** based on the requirement we will be choosing the most appropriate nonlinear [activation function](#) to produce the better result. Now let us implement a single-layer perceptron.

Architecture

There exist five basic types of neuron connection architecture :

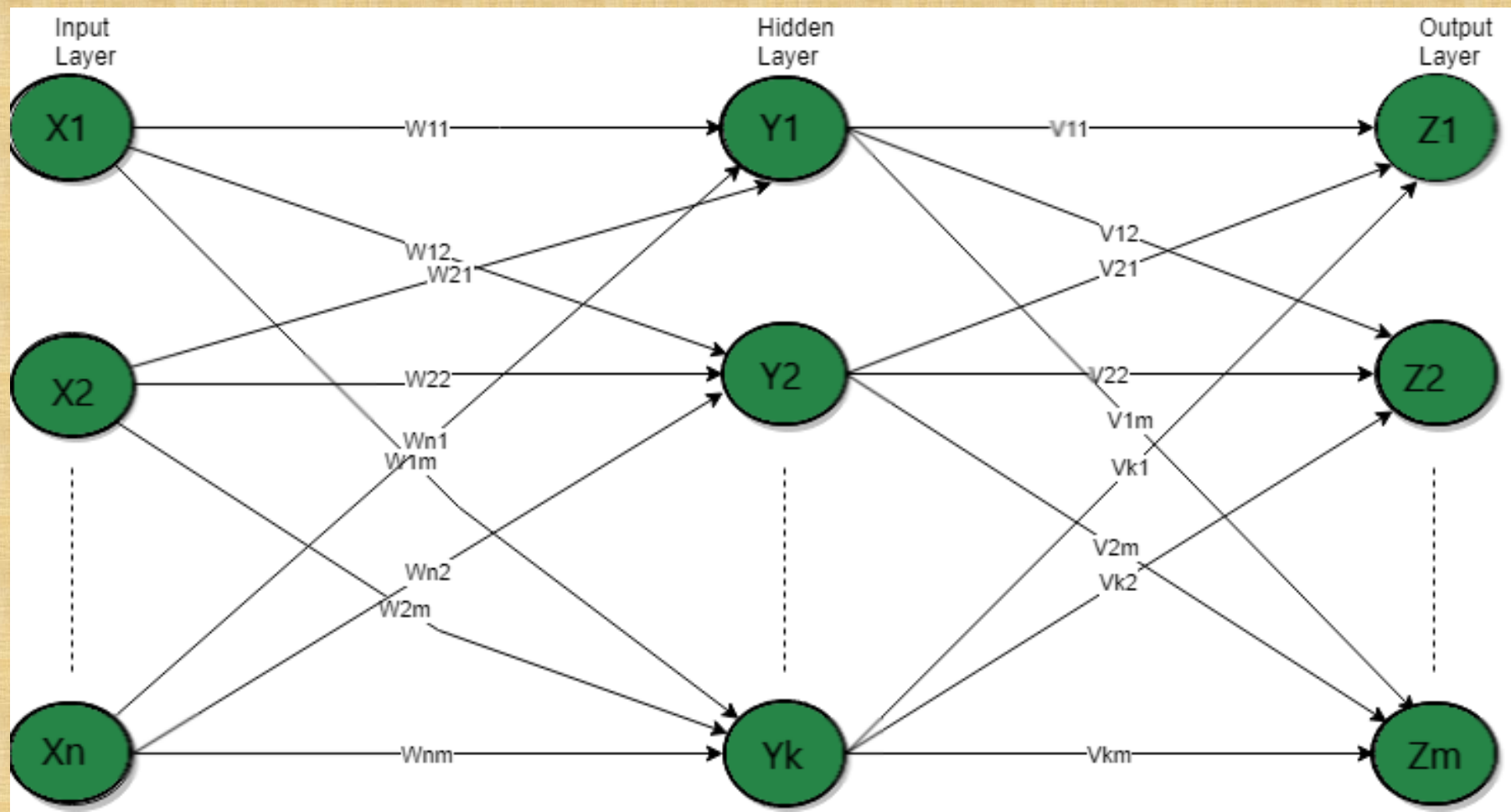
- Single-layer feed-forward network
- Multilayer feed-forward network
- Single node with its own feedback
- Single-layer recurrent network
- Multilayer recurrent network

Single-layer feed-forward network



In this type of network, we have **only two layers** **input layer** and the **output layer** but the **input layer does not count because no computation is performed in this layer**. The **output layer is formed when different weights are applied to input nodes** and the cumulative effect per node is taken. After this, the neurons collectively give the output layer to compute the output signals.

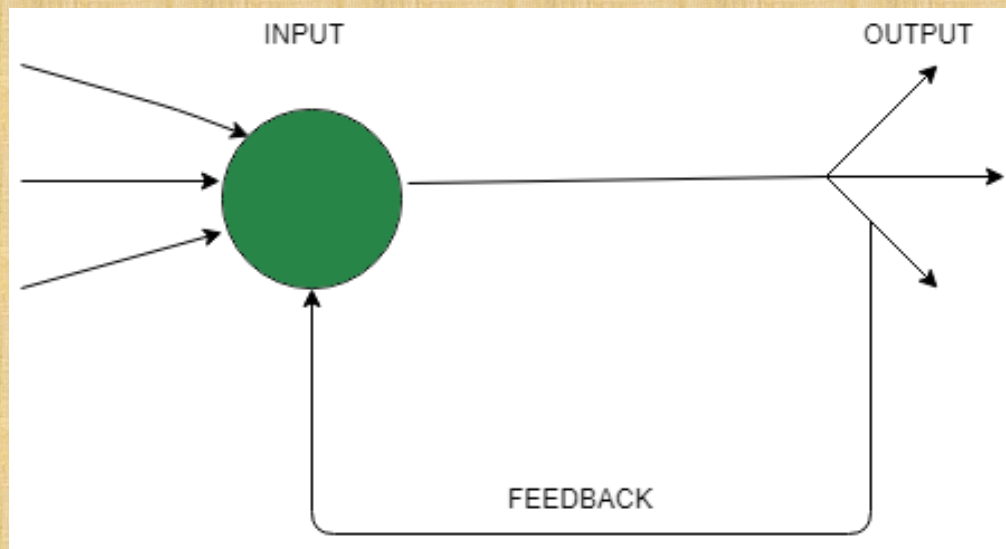
Multilayer feed-forward network



Multilayer feed-forward network

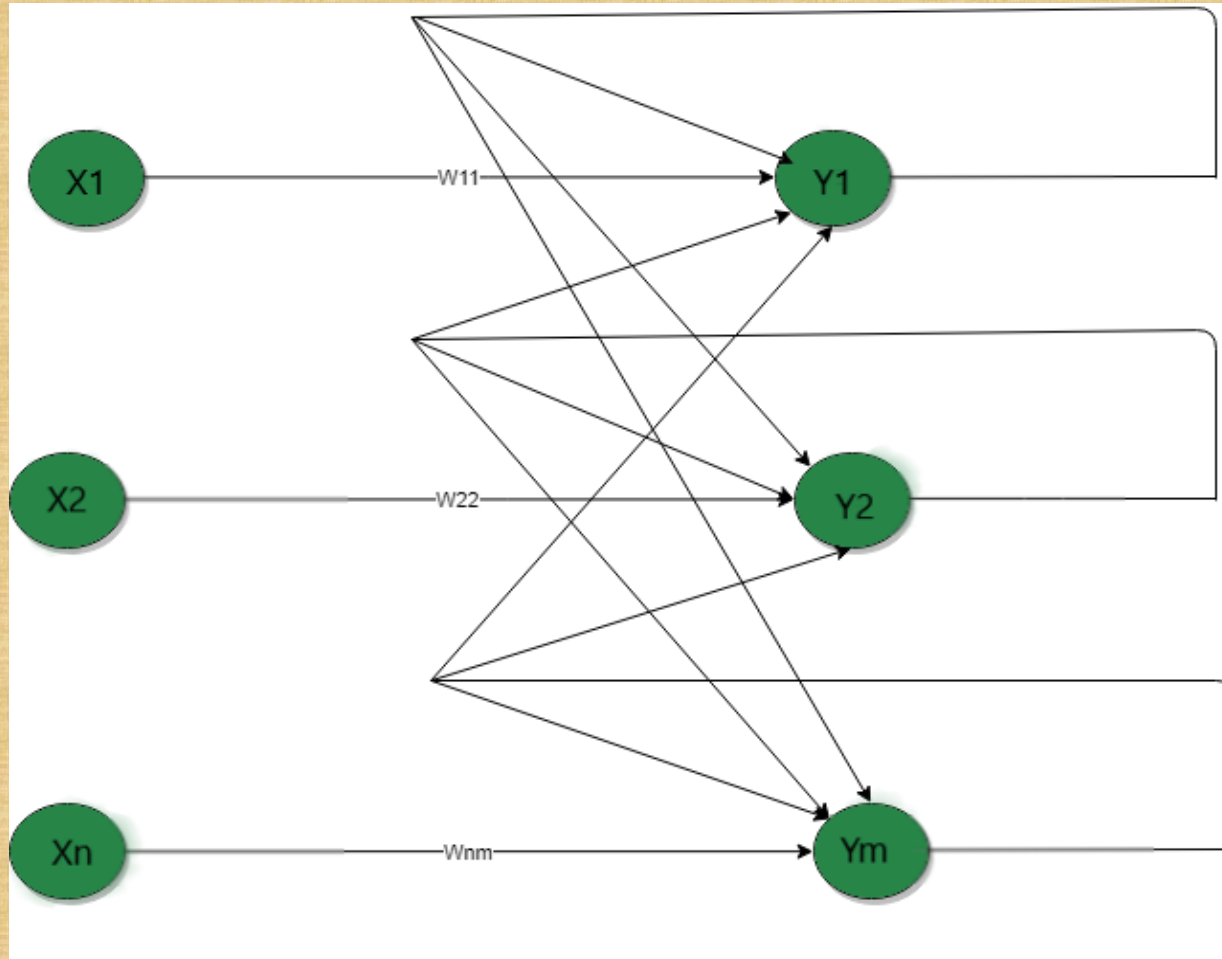
This layer also has a **hidden layer that is internal to the network** and has no direct contact with the external layer. The existence of one or more hidden layers enables the network to be computationally stronger, a feed-forward network because of information flow through the input function, and the intermediate computations used to determine the output Z . There are **no feedback connections** in which outputs of the model are fed back into itself.

Single node with its own feedback



When outputs can be directed back as inputs to the same layer or preceding layer nodes, then it results in feedback networks. Recurrent networks are feedback networks with closed loops. The above figure shows a single recurrent network having a single neuron with feedback to itself.

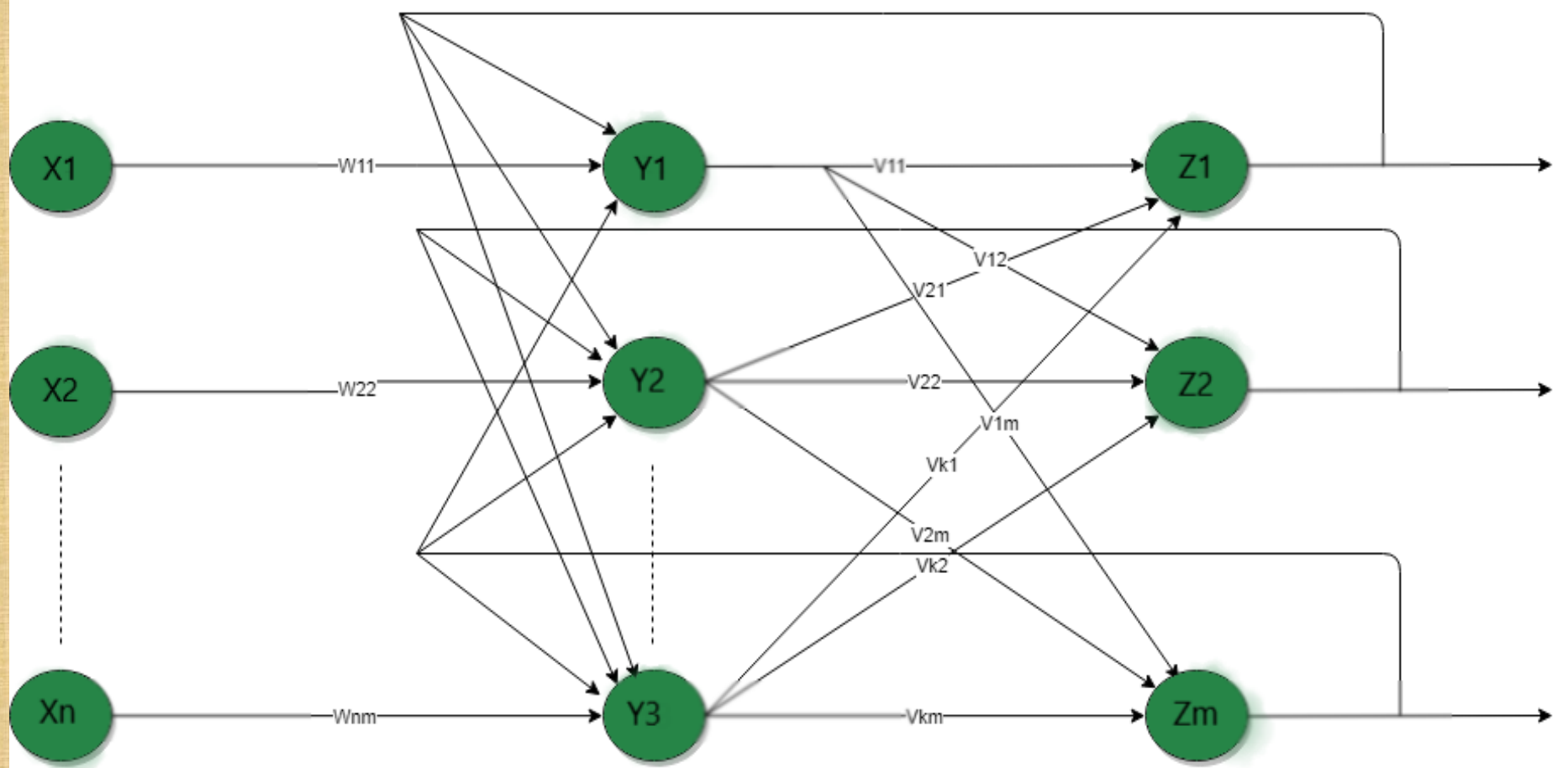
Single-layer recurrent network



Single-layer recurrent network

The above network is a single-layer network with a feedback connection in which the processing element's output can be directed **back to itself** or to another processing element or both. A recurrent neural network is a class of artificial neural networks where connections between nodes form a directed graph along a sequence. This allows it to exhibit dynamic temporal behavior for a time sequence. Unlike feedforward neural networks, RNNs can use their internal state (memory) to process sequences of inputs.

Multilayer recurrent network



Multilayer recurrent network

In this type of network, processing element output can be directed to the processing element in the same layer and in the preceding layer forming a multilayer recurrent network. They perform the same task for every element of a sequence, with the output being dependent on the previous computations. Inputs are not needed at each time step. The main feature of a Recurrent Neural Network is its hidden state, which captures some information about a sequence.

What is a Neural Network?

A neural network is a **computational model inspired by the human brain's network of neurons**. It consists of layers of **interconnected nodes (neurons)** that process input data to produce an output. Neural networks are used in various **applications, from image and speech recognition to natural language processing and autonomous systems.**

Types of Neural Networks

Neural networks classified into two categories:

Feedforward Neural Networks (FNNs): These networks have a unidirectional flow of information from input to output, with no cycles or loops. They are typically used for tasks like image classification and regression.

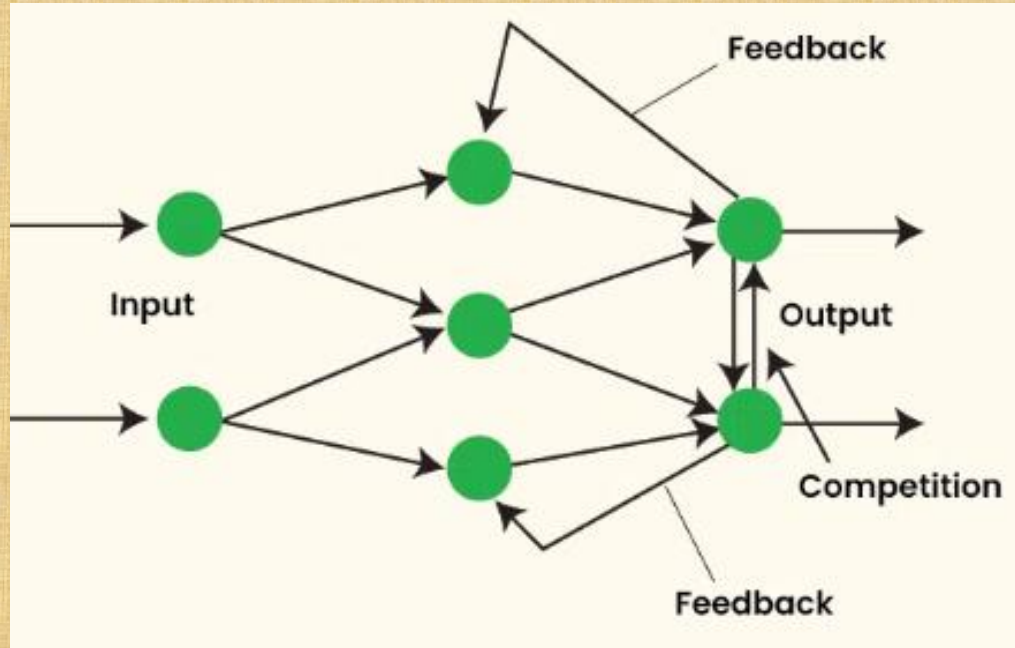
Feedback Neural Networks (RNNs): These networks have connections that loop back, allowing information to be fed back into the network. This structure enables them to handle sequential data and temporal dependencies, making them suitable for tasks like time series prediction and language modeling.

Structure of Feedback Neural Networks

Feedback neural networks, or RNNs, are characterized by their ability to maintain a state that captures information about previous inputs. This is achieved through recurrent connections that loop back from the output to the input of the same layer or previous layers. The key components of an RNN include:

- **Input Layer:** Receives the input data.
- **Hidden Layers:** Contain neurons with recurrent connections that maintain a state over time.
- **Output Layer:** Produces the final output based on the processed information.

Structure of Feedback Neural Networks



The recurrent connections allow RNNs to maintain a memory of previous inputs, which is crucial for tasks involving sequential data.

Mechanisms of Feedback in Neural Networks

Backpropagation: Backpropagation is a method of feedback that involves the computation of the error gradient at each layer of the network. The error gradient is then used to update the network's parameters. Backpropagation is widely used in deep neural networks due to its efficiency and accuracy.

Recurrent Connections: Recurrent connections involve the feedback of information from a later stage of the network to an earlier stage. This type of feedback is used in recurrent neural networks (RNNs), which are designed to handle sequential data.

Lateral Connections: Lateral connections involve the feedback of information between neurons in the same layer. This type of feedback is used in applications such as image processing, where the goal is to capture spatial relationships between pixels.

Hopfield Neural Network

The Hopfield Neural Networks, invented by Dr John J. Hopfield consists of one layer of ' n ' fully connected recurrent neurons. It is generally used in performing auto-association and optimization tasks. It is calculated using a converging interactive process and it generates a different response than our normal neural nets.

Discrete Hopfield Network

It is a fully interconnected [neural network](#) where each unit is connected to every other unit. It behaves in a discrete manner, i.e. it gives finite distinct output, generally of two types:

Binary (0/1)

Bipolar (-1/1)

Hopfield Network

- Hopfield networks are a **type of recurrent neural network** with fully connected neurons.
- They are used for **auto-association and optimization problems**.
- Hopfield networks have **symmetric connections** and use an energy function to converge to stable states, representing stored patterns.
- They are **trained using the Hebbian learning rule** and can be used to reconstruct patterns from noisy or partial inputs.

Hopfield Neural Network

The weights associated with this network are symmetric in nature and have the following properties.

$$1. w_{ij} = w_{ji}$$

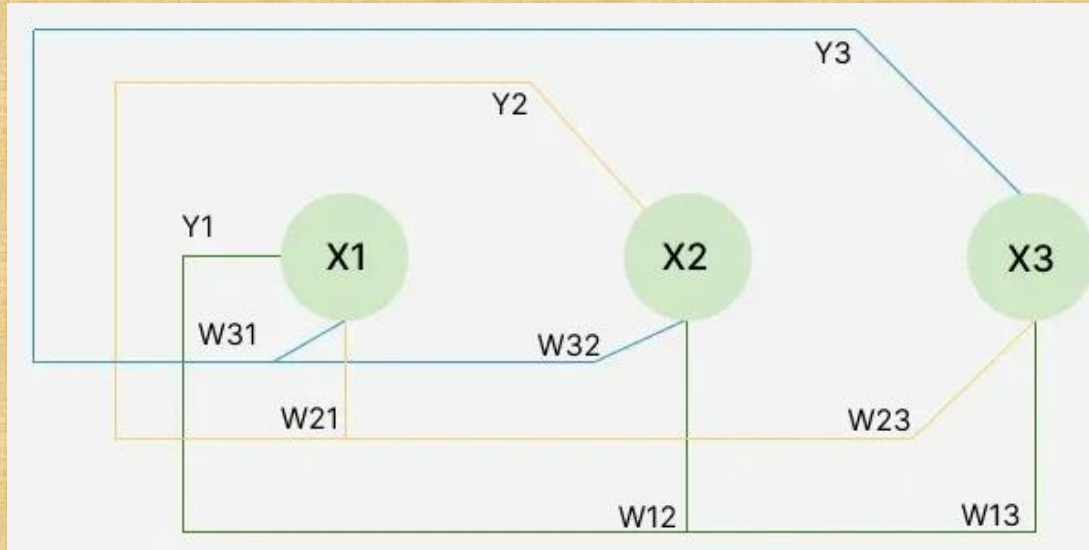
$$2. w_{ii} = 0$$

Structure & Architecture of Hopfield Network

- Each neuron has an inverting and a non-inverting output.
- Being fully connected, the output of each neuron is an input to all other neurons but not the self.

Hopfield Neural Network

The below figure shows a sample representation of a Discrete Hopfield Neural Network architecture having the following elements.



$[x_1, x_2, \dots, x_n] \rightarrow$ Input to the n given neurons.

$[y_1, y_2, \dots, y_n] \rightarrow$ Output obtained from the n given neurons

$W_{ij} \rightarrow$ weight associated with the connection between the i^{th} and the j^{th} neuron.

Hopfield Neural Network- Training Algorithm

For storing a set of input patterns $S(p)$ [$p = 1$ to P], where $S(p) = S_1(p) \dots S_i(p) \dots S_n(p)$, the weight matrix is given by:

- **For binary patterns**

$$w_{ij} = \sum_{p=1}^P [2s_i(p) - 1][2s_j(p) - 1] \text{ (} w_{ij} \text{ for all } i \neq j \text{)}$$

- **For bipolar patterns**

$$w_{ij} = \sum_{p=1}^P [s_i(p)s_j(p)] \text{ (where } w_{ij} = 0 \text{ for all } i = j \text{)}$$

(i.e. weights here have no self-connection)

Hopfield Neural Network- Training Algorithm

Steps Involved in the training of a Hopfield Network are as mapped below:

- Initialize weights (w_{ij}) to store patterns (using training algorithm).
- For each input vector y_i , perform **steps 3-7**.
- Make the initial activators of the network equal to the external input vector x .

$$y_i = x_i : (for\ i = 1\ to\ n)$$

- For each vector y_i , perform **steps 5-7**.
- Calculate the total input of the network y_{in} using the equation given below.

$$y_{in_i} = x_i + \sum_j [y_j w_{ji}]$$

Hopfield Neural Network- Training Algorithm

- Apply activation over the total input to calculate the output as per the equation given below:

$$y_i = \begin{cases} 1 & \text{if } y_{in} > \theta_i \\ y_i & \text{if } y_{in} = \theta_i \\ 0 & \text{if } y_{in} < \theta_i \end{cases}$$

(where θ_i (threshold) and is normally taken as 0)

- Now feedback the obtained output y_i to all other units. Thus, the activation vectors are updated.
- Test the network for convergence.

Bidirectional Associative Memory (BAM)

Bidirectional Associative Memory (BAM) is a [supervised learning](#) model in Artificial Neural Network. This is *hetero-associative memory*, for an input pattern, it returns another pattern which is potentially of a different size. This phenomenon is very similar to the human brain. Human memory is necessarily associative. It uses a chain of mental associations to recover a lost memory like associations of faces with names, in exam questions with answers, etc. In such memory associations for one type of object with another, a [Recurrent Neural Network \(RNN\)](#) is needed to receive a pattern of one set of neurons as an input and generate a related, but different, output pattern of another set of neurons.

Bidirectional Associative Memory (BAM)

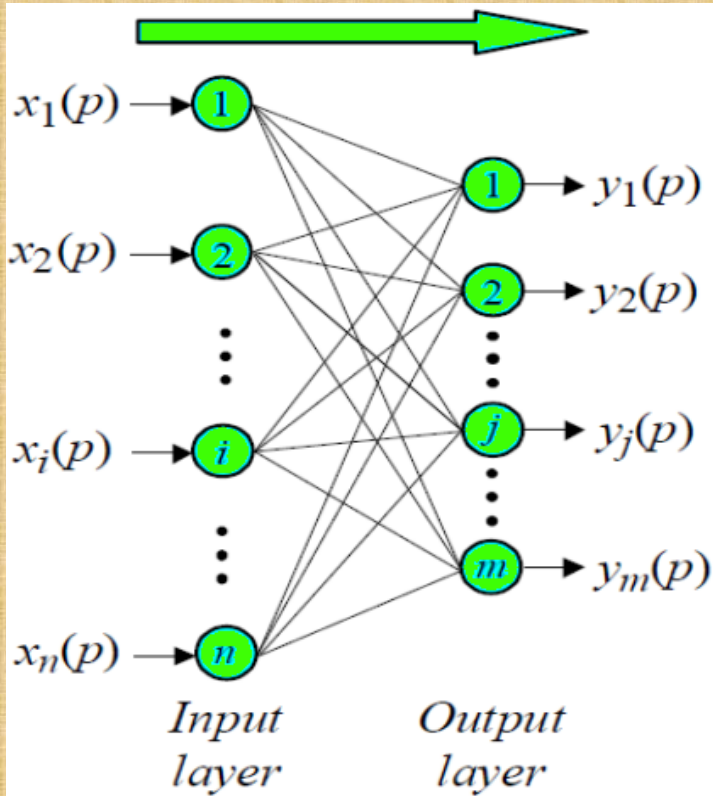
Why BAM is required?

The main objective to introduce such a network model is to store hetero-associative pattern pairs. This is used to retrieve a pattern given a noisy or incomplete pattern. **BAM Architecture:** When BAM accepts an input of n -dimensional vector X from set A then the model recalls m -dimensional vector Y from set B . Similarly when Y is treated as input, the BAM recalls X .

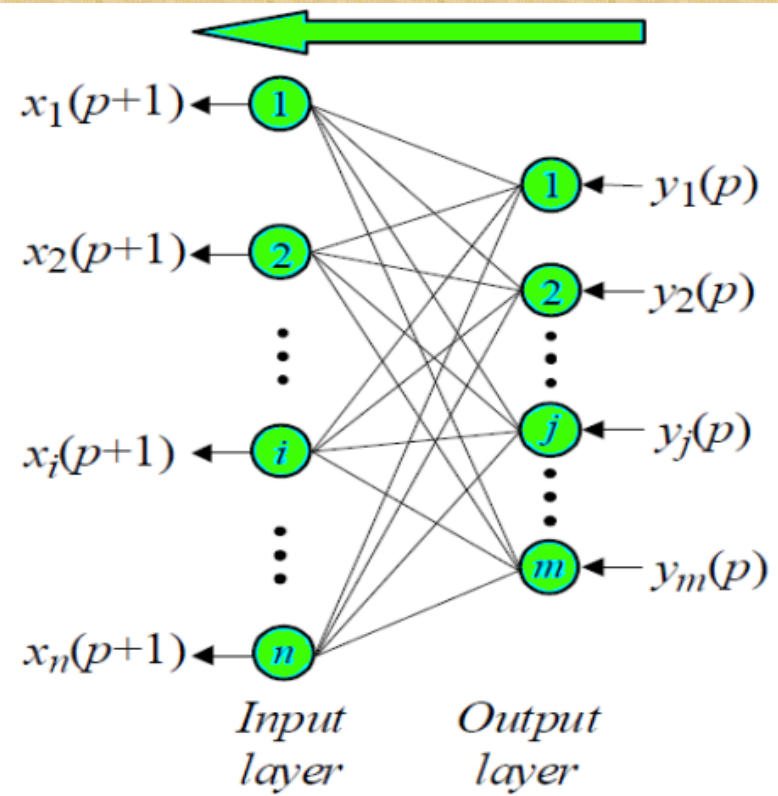
Bidirectional Associative Memory (BAM)

- BAM is a type of neural network that can **associate patterns in both forward and backward directions**.
- It consists of **two layers**: an **input layer** and an **output layer**, with connections between them.
- It is used for **pattern recognition and recall tasks**, where it can retrieve an **associated pattern** when presented with a partial or noisy input.
- BAM networks are **trained using the Hebbian learning rule**, which strengthens connections between neurons that are simultaneously active.

ANN - Bidirectional Associative Memory (BAM)



(a) Forward direction.



(b) Backward direction.

ANN - Bidirectional Associative Memory (BAM)

Algorithm:

1. **Storage (Learning):** In this learning step of BAM, weight matrix is calculated between M pairs of patterns (fundamental memories) are stored in the synaptic weights of the network following the equation
$$W = \sum_{m=1}^M X_m Y_m^T$$
2. **Testing:** We have to check that the BAM recalls perfectly Y_m for corresponding X_m and recalls X_m for corresponding Y_m . Using,
$$Y_m = \operatorname{sign}(W^T X_m), \quad m=1, 2, \dots, M$$
$$X_m = \operatorname{sign}(W Y_m), \quad m=1, 2, \dots, M$$
 All pairs should be recalled accordingly.

ANN - Bidirectional Associative Memory (BAM)

3. **Retrieval:** For an unknown vector X (a corrupted or incomplete version of a pattern from set A or B) to the BAM and retrieve a previously stored association: $X \neq$

$$X_m, \quad m = 1, 2, \dots, M$$

- **Initialize the BAM:** $[Tex] \backslash [X(0)=X, \quad p=0] \backslash [Tex]$
- **Calculate the BAM output at iteration p :** $[Tex] \backslash [Y(p)=\operatorname{sign}\left[W^T X(p)\right]] \backslash [Tex]$
- **Update the input vector $X(p)$:** $[Tex] \backslash [X(p+1)=\operatorname{sign}[W Y(p)]] \backslash [Tex]$
- **Repeat** the iteration until convergence, when input and output remain unchanged.

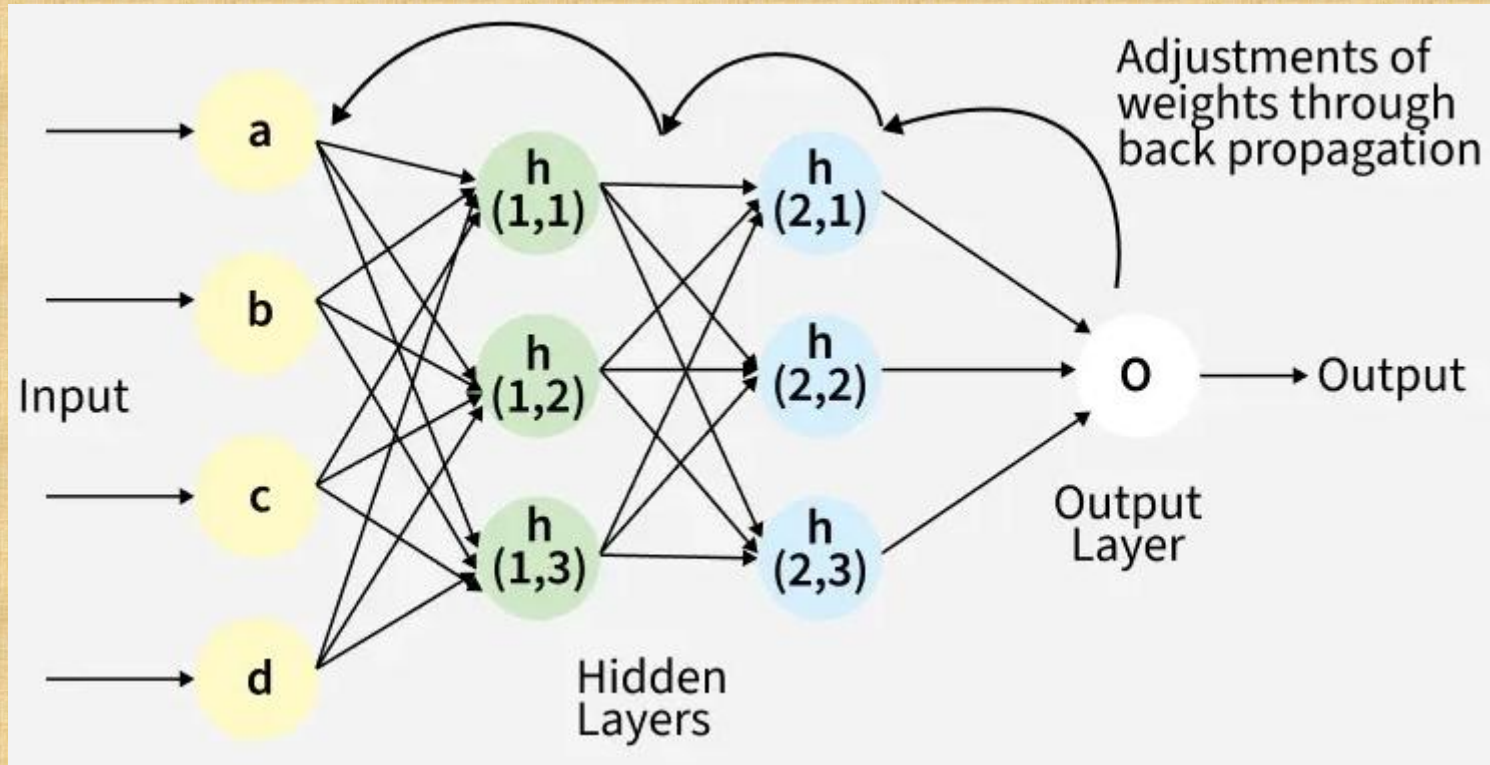
Limitations of BAM:

- **Storage capacity of the BAM:** In the BAM, stored number of associations should not be exceeded the number of neurons in the smaller layer.
- **Incorrect convergence:** Always the closest association may not be produced by BAM.

Back Propagation Network (BPN)

- Backpropagation, short for **Backward Propagation of Errors**, is a key algorithm used to train neural networks by **minimizing the difference between predicted and actual outputs**.
- It works by **propagating errors backward through the network**, **using the chain rule of calculus** to **compute gradients** and then **iteratively updating the weights and biases**. Combined with optimization techniques like gradient descent, backpropagation enables the model to reduce loss across epochs and effectively learn complex patterns from data.

Back Propagation Network (BPN)



Back Propagation Network (BPN)

Back Propagation plays a critical role in how neural networks improve over time.

- **Efficient Weight Update:** It computes the gradient of the loss function with respect to each weight using the chain rule making it possible to update weights efficiently.
- **Scalability:** The Back Propagation algorithm scales well to networks with multiple layers and complex architectures making deep learning feasible.
- **Automated Learning:** With Back Propagation the learning process becomes automated and the model can adjust itself to optimize its performance.

Working of Back Propagation Algorithm

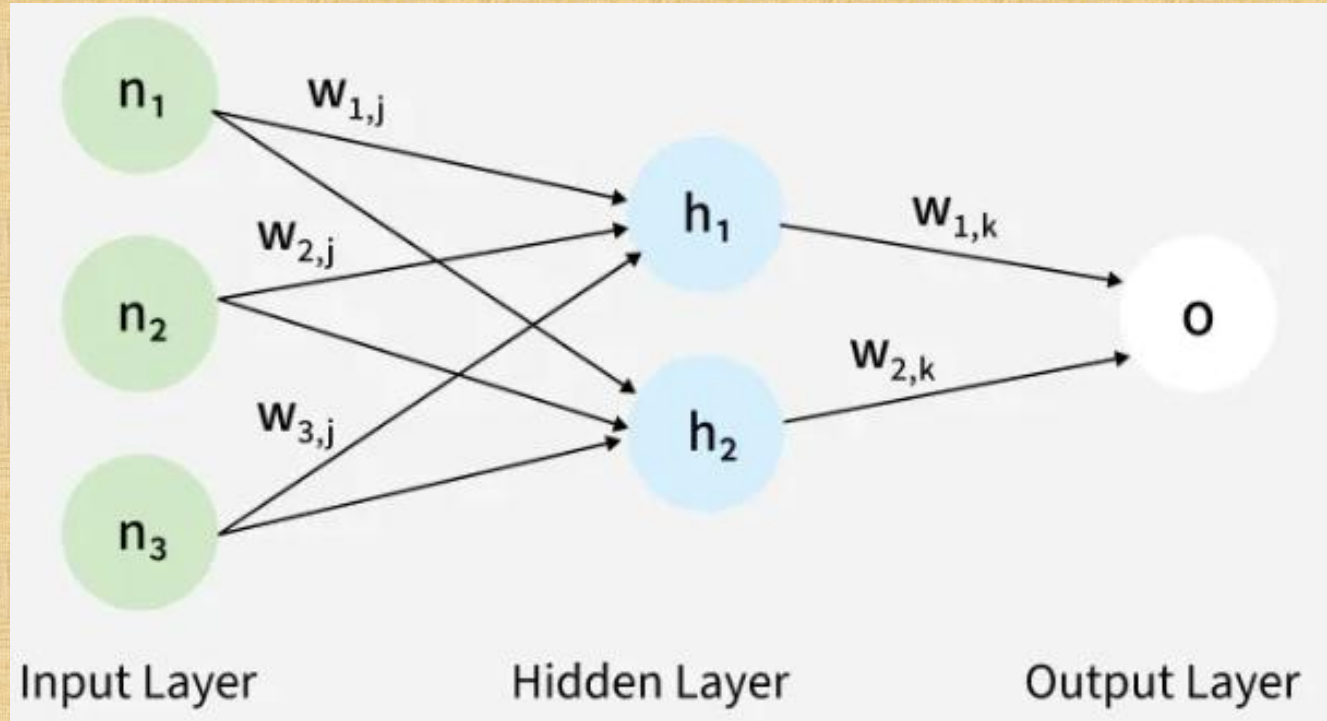
The Back Propagation algorithm involves **two main steps**: the Forward Pass and the Backward Pass.

1. Forward Pass Work

In forward pass the input data is fed into the input layer. These inputs combined with their respective weights are passed to hidden layers. For example in a network with two hidden layers (h1 and h2) the output from h1 serves as the input to h2. Before applying an activation function, a bias is added to the weighted inputs.

Each hidden layer computes the weighted sum ($\sum$) of the inputs then applies an activation function like [ReLU \(Rectified Linear Unit\)](#) to obtain the output (σ). The output is passed to the next layer where an activation function such as [softmax](#) converts the weighted outputs into probabilities for classification.

Working of Back Propagation Algorithm



The forward pass using weights and biases

Working of Back Propagation Algorithm

2. Backward Pass

In the backward pass the error (the difference between the predicted and actual output) is propagated back through the network to adjust the weights and biases. One common method for error calculation is the [Mean Squared Error \(MSE\)](#) given by:

$$\text{MSE} = (\text{Predicted Output} - \text{Actual Output})^2$$

Once the error is calculated the network adjusts weights using gradients which are computed with the chain rule. These gradients indicate how much each weight and bias should be adjusted to minimize the error in the next iteration. The backward pass continues layer by layer ensuring that the network learns and improves its performance. The activation function through its derivative plays a crucial role in computing these gradients during Back Propagation.

Radial Basis Function Network (RBFN)

Radial Basis Function (RBF) Neural Networks are used for function approximation tasks. They are a special category of feed-forward neural networks comprising of three layers. Due to this distinct three-layer architecture and universal approximation capabilities they offer faster learning speeds and efficient performance in classification and regression problems.

How Do RBF Networks Work?

RBF Networks are conceptually similar to [K-Nearest Neighbor \(k-NN\)](#) models though their implementation is distinct. The fundamental idea is that nearby items with similar predictor variable values influence an item's predicted target value. The RBF Networks operates as:

Input Vector: The network receives an n-dimensional input vector that needs classification or regression.

RBF Neurons: Each neuron in the hidden layer represents a prototype vector from the training set. The network computes the Euclidean distance between the input vector and each neuron's center.

Activation Function: The Euclidean distance is transformed using a [Radial Basis Function](#) (typically a Gaussian function) to compute the neuron's activation value. This value decreases exponentially as the distance increases.

Output Nodes: Each output node calculates a score based on a weighted sum of the activation values from all RBF neurons. For classification the category with the highest score is chosen.

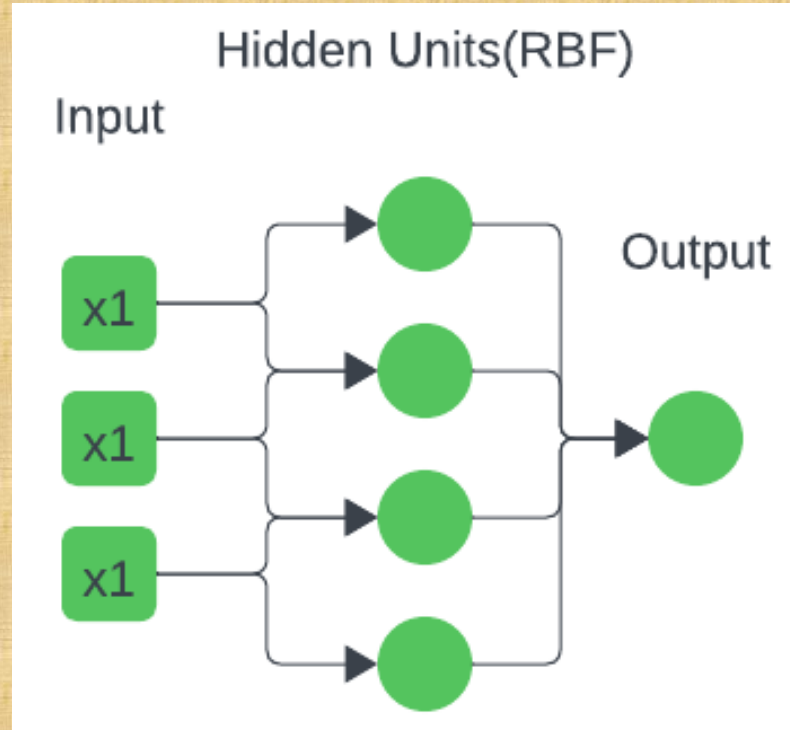
How Do RBF Networks Work?

Example: Consider a dataset with two-dimensional data points from two classes. An RBF Network trained with 20 neurons will have each neuron representing a prototype in the input space. The network computes category scores which can be visualized using 3-D mesh or contour plots. We assign positive weights to neurons in the same category and negative weights to neurons in different categories. The decision boundary can be plotted by evaluating scores over a grid.

Key Characteristics of RBFs

- **Radial Basis Functions:** These are real-valued functions dependent solely on the distance from a central point. The Gaussian function is the most commonly used type.
- **Dimensionality:** The network's dimensions correspond to the number of predictor variables.
- **Center and Radius:** Each RBF neuron has a center and a radius (spread). The radius affects how broadly each neuron influences the input space.

Architecture of RBF Networks



Architecture of RBF Networks

The architecture of an RBF Network typically consists of **three layers**:

1. Input Layer

Function: After receiving the input features the input layer sends them straight to the hidden layer.

Components: It is made up of the same number of neurons as the characteristics in the input data. One feature of the input vector corresponds to each neuron in the input layer.

Architecture of RBF Networks

2. Hidden Layer

Function: This layer uses radial basis functions (RBFs) to conduct the non-linear transformation of the input data.

Components: Neurons in the buried layer apply the RBF to the incoming data. The Gaussian function is the RBF that is most frequently utilized.

RBF Neurons: Every neuron in the hidden layer has a spread parameter (σ) and a center which are also referred to as prototype vectors. The spread parameter modulates the distance between the center of an RBF neuron and the input vector which in turn determines the neuron's output.

Architecture of RBF Networks

3. Output Layer

Function: The output layer uses weighted sums to integrate the hidden layer neurons outputs to create the network's final output.

Components: It is made up of neurons that combine the outputs of the hidden layer in a linear fashion. To reduce the error between the network's predictions and the actual target values, the weights of these combinations are changed during training.

Implementing Radial Basis Function Neural Network

An RBF neural network must be trained in **three stages**: choosing the center's, figuring out the spread parameters and training the output weights.

Step 1: Selecting the Centers

Techniques for Centre Selection: Centre's can be picked at random from the training set of data or by applying techniques such as **k-means clustering**

K-Means Clustering: The center's of these clusters are employed as the center's for the RBF neurons in this widely used center selection technique which groups the input data into k groups.

Implementing Radial Basis Function Neural Network

Step 2: Determining the Spread Parameters

The spread parameter (σ) governs each RBF neuron's area of effect and establishes the width of the RBF.

Calculation: The spread parameter can be manually adjusted for each neuron or set as a constant for all neurons. Setting σ based on the separation between the center's is a popular method, frequently accomplished with the help of a heuristic like dividing the greatest distance between centers by the square root of twice the number of center's

Implementing Radial Basis Function Neural Network

Step 3: Training the Output Weights

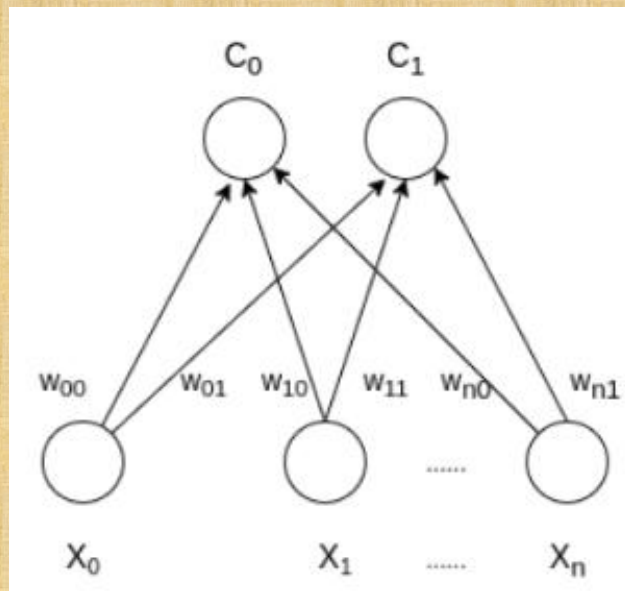
Linear Regression: The objective of [linear regression](#) techniques which are commonly used to estimate the output layer weights, is to minimize the error between the anticipated output and the actual target values.

Pseudo-Inverse Method: One popular technique for figuring out the weights is to utilize the pseudo-inverse of the hidden layer outputs matrix

Self Organizing Maps - Kohonen Maps

- A **Self Organizing Map (SOM)** or **Kohonen Map** is an **unsupervised neural network algorithm** based on biological neural models from the 1970s.
- It uses a competitive learning approach and is primarily designed for clustering and dimensionality reduction.
- SOM effectively maps high-dimensional data to a lower-dimensional grid enabling easier interpretation and visualization of complex datasets.

Self Organizing Maps - Kohonen Maps



It consists of two primary layers:

- **Input Layer:** Represents the features of the data.
- **Output Layer:** Arranged as a 2D grid of neurons with each neuron representing a cluster in the data.

Working of Self Organizing Maps (SOM)

1. Initialization

The weights of the output neurons are randomly initialized. These weights represent the features of each neuron and will be adjusted during training.

2. Competition

For each input vector, SOM computes the Euclidean distance between the input and the weight vectors of all neurons. The neuron with the smallest distance is the winning neuron.

Formula : $D(j) = \sum_{i=1}^n (w_{ij} - x_i)^2$

where

- $D(j)$ is the distance for neuron
- j and n is the number of features.

Working of Self Organizing Maps (SOM)

3. Weight Update

The winning neuron's weights are updated to move closer to the input vector. The weights of neighboring neurons are also adjusted but with smaller changes.

$$\text{Formula: } w_{ij}^{(new)} = w_{ij}^{(old)} + \alpha \cdot (x_i - w_{ij}^{(old)})$$

where

- α is the learning rate
- x_i is the input feature.

4. Learning Rate Decay

The learning rate α decreases over time allowing the map to converge to stable values.

$$\text{Formula: } \alpha(t + 1) = 0.5 \cdot \alpha(t)$$

Working of Self Organizing Maps (SOM)

5. Stopping Condition

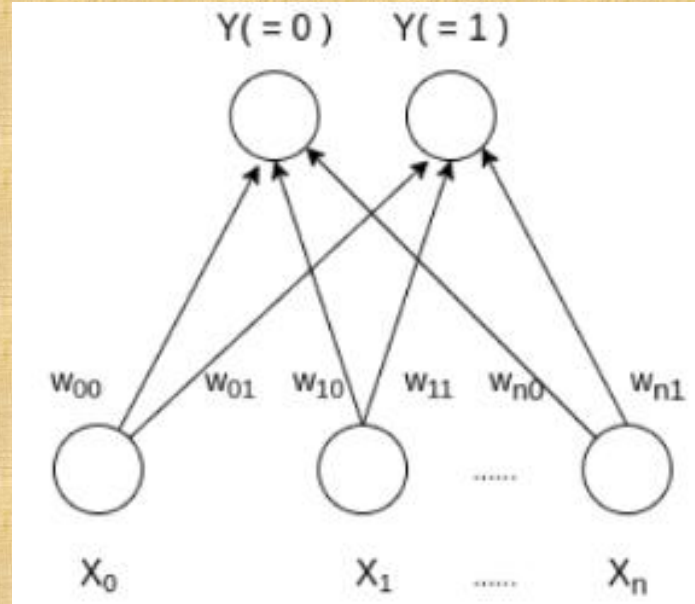
The training stops when the maximum number of epochs is reached or when the weights converge.

Learning Vector Quantization (LVQ)

Learning Vector Quantization (LVQ) is a type of artificial neural network that's inspired by how our brain processes information. It's a supervised classification algorithm that uses a prototype-based approach. LVQ learns by selecting representative vectors (called codebooks or weights) and adjusts them during training to best represent different classes.

LVQ has two layers, one is the Input layer and the other one is the Output layer. The architecture of the Learning Vector Quantization with the number of classes in an input data and " n " number of input features for any sample is given below:

Learning Vector Quantization (LVQ)



How Learning Vector Quantization works?

- 1. Pick starting points:** Choose one example from each class to act as a starting point called a weight. These are used to represent each class.
- 2. For each training example:** Measure how far it is from each weight using distance . Find the closest weight that's the one we'll update.
- 3. Update the closest weight:** If it's the correct class move the weight closer to the training example.

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha(t) * (x_{ik} - w_{ij}(\text{old}))$$

How Learning Vector Quantization works?

If it's the wrong class push the weight away from the training example.

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) - \alpha(t) * (x_{ik} - w_{ij}(\text{old}))$$

- w_{ij} = weight vector for class j
- x_{ik} = feature value from the training example
- $\alpha(t)$ = learning rate at time t (a small value that reduces over time)

4. Repeat: Keep going through all training examples multiple times. The weights slowly adjust to better match their classes.

LVQ: Algorithm

Step 1: Initialize reference vectors.

Select Initial Weights: Pick one or more samples from the training set to act as prototype vectors (weights). Each should represent a different class.

Assign Labels: Give each weight the class label of the sample it was taken from.

Step 2: Compute Distance

For each input sample x , calculate the Euclidean distance to all weight vectors w_j .

$$D(j) = \sum (x_i - w_{ij})^2$$

Winner = weight with minimum distance also called the Best Matching Unit (BMU)

LVQ: Algorithm

Step 3: Update Weights

If the winning weight's class matches the sample's label:

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha(t) * (x_{ik} - w_{ij}(\text{old}))$$

If it's the wrong class:

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) - \alpha(t) * (x_{ik} - w_{ij}(\text{old}))$$

Step 4: Repeat for Multiple Epochs

- Iterate over the training set multiple times.
- Optionally decay the learning rate over epochs for finer tuning.
- Weights gradually shift to better represent their classes.

Learning of neural networks:

- ✓ Concept of learning
 - Learning in
 - ✓ Single layer feed forward neural network
 - Multilayer feed forward neural network
 - Recurrent neural network
 - Types of learning in neural networks

Concept of Learning

The concept of learning

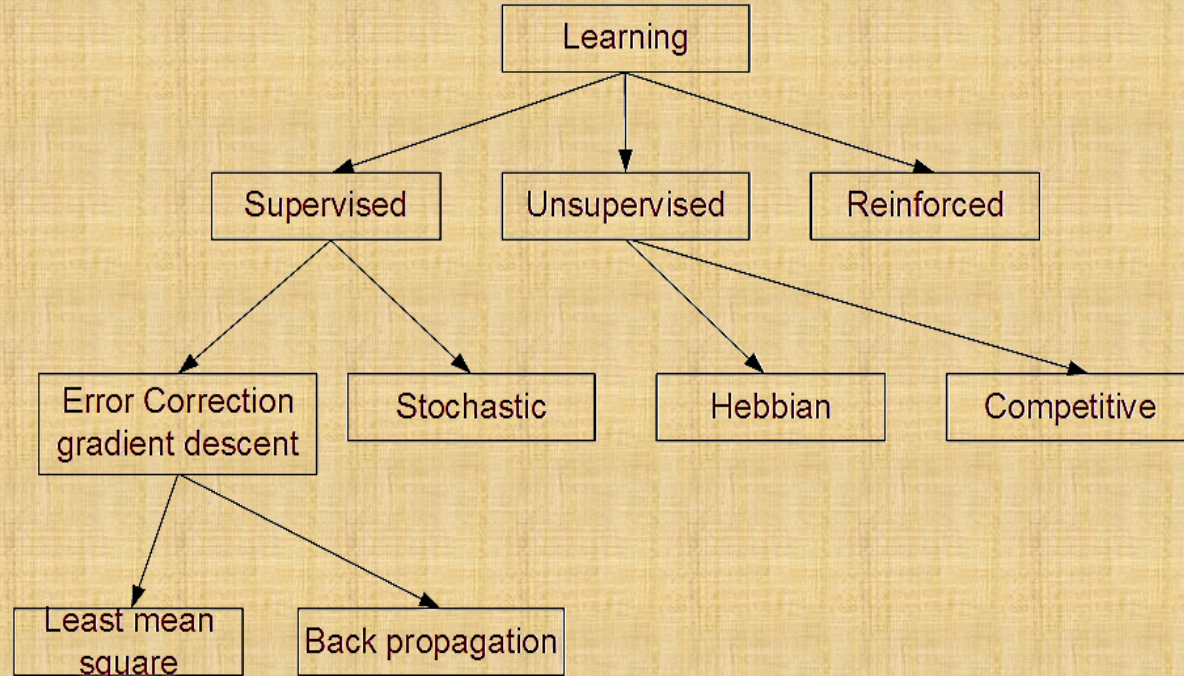
- The learning is an important feature of **human computational ability**.
- Learning may be viewed as the **change in behaviour acquired due to practice or experience**.
- As it occurs, the **effective coupling between the neuron is modified**.

The concept of learning

- In case of artificial neural networks, it is a **process of modifying neural network by updating its weights, biases and other parameters, if any.**
- During the learning, **the parameters of the networks are optimized** and as a result process of curve fitting.
- It is then said that the **network has passed** through a learning Phase.

Types of learning

- There are several learning techniques.



Supervised learning

- In this learning, every input pattern that is used to train the network is associated with an output pattern.
- This is called training set of data. Thus, in this form of learning, the input-output relationship of the training scenarios are available.
- Here, the output of a network is compared with the corresponding target value and the error is determined.
- It is then feed back to the network for updating the same. This results in an improvement.
- This type of training is called learning with the help of teacher.

Unsupervised learning

- If the **target output is not available**, then the **error in prediction can not be determined** and in such a situation, the system learns of **its own** by **discovering and adapting to structural features in the input patterns**.
- This type of training is called **learning without a teacher**.

Reinforced learning

- In this techniques, although a teacher is available, it **does not tell the expected answer**, but **only tells if the computed output is correct or incorrect**. A reward is given for a correct answer computed and a penalty for a wrong answer. This information helps the network in its learning process.
- **Note:** Supervised and unsupervised learnings are the most popular forms of learning. Unsupervised learning is very common in biological systems.
- It is also important for artificial neural networks: **training data are not always available** for the intended application of the neural network.

Training SLFFNNs

Single layer feed forward NN (SLFFNN) training

- We know that, several neurons are arranged in one layer with inputs and weights connect to every neuron.
- Learning in such a network occurs by adjusting the weights associated with the inputs so that the network can classify the input patterns.
- A single neuron in such a neural network is called **perceptron**.

Single layer feed forward NN training

The algorithm to **train a perceptron** is stated below.

- Let there is a perceptron with $(n + 1)$ inputs $x_0, x_1, x_2, \dots, x_n$ where $x_0 = 1$ is the **bias input**.
- Let f denotes the transfer function of the neuron. Suppose, $\bar{X}$ and $\bar{Y}$ denotes the input-output vectors as a training data set. $\bar{W}$ denotes the weight matrix.

With this input-output relationship pattern and configuration of a perceptron, the algorithm **Training Perceptron** to train the perceptron is stated in the following slide.

Single layer feed forward NN training

1. Initialize $W = w_0, w_1, \dots, w_n$ to some **random weights**.
2. For each input pattern $x \in \bar{X}$ do **Here, $x = \{x_0, x_1, \dots, x_n\}$**
 - Compute $I = \sum_{i=0}^n w_i x_i$
 - Compute observed output y

$$y = f(I) = \begin{cases} 1, & \text{if } I > 0 \\ 0, & \text{if } I \leq 0 \end{cases}$$

$\bar{Y} = \bar{Y}' + y$ **Add y to $\bar{Y}'$, which is initially empty**

Single layer feed forward NN training

3. If the desired output $\bar{Y}$ matches the observed output $\bar{Y}'$ then output occurs.
4. Otherwise, update the weight matrix $\bar{W}$ as follows :
 - For each output $y \in \bar{Y}'$ do
 - If the observed out y is 1 instead of 0, then
$$w_i = w_i - \alpha x_i, (i = 0, 1, 2, \dots, n)$$
 - Else, if the observed out y is 0 instead of 1, then
$$w_i = w_i + \alpha x_i, (i = 0, 1, 2, \dots, n)$$
$$\alpha \text{ is the learning parameter}$$
5. Go to step 2.

Single layer feed forward NN training

In the above algorithm, α is the learning parameter and is a constant decided by some empirical studies.

Note :

- The algorithm **Training Perceptron** is based on the supervised learning technique.
- **ADALINE: Adaptive Linear Network Element** is also an alternative term to perceptron.
- If there are 10 number of neurons in the single layer feed forward neural network to be trained, then we have to iterate the algorithm 10 times for each perceptron in the network.

Learning of neural networks:

✓ Learning in

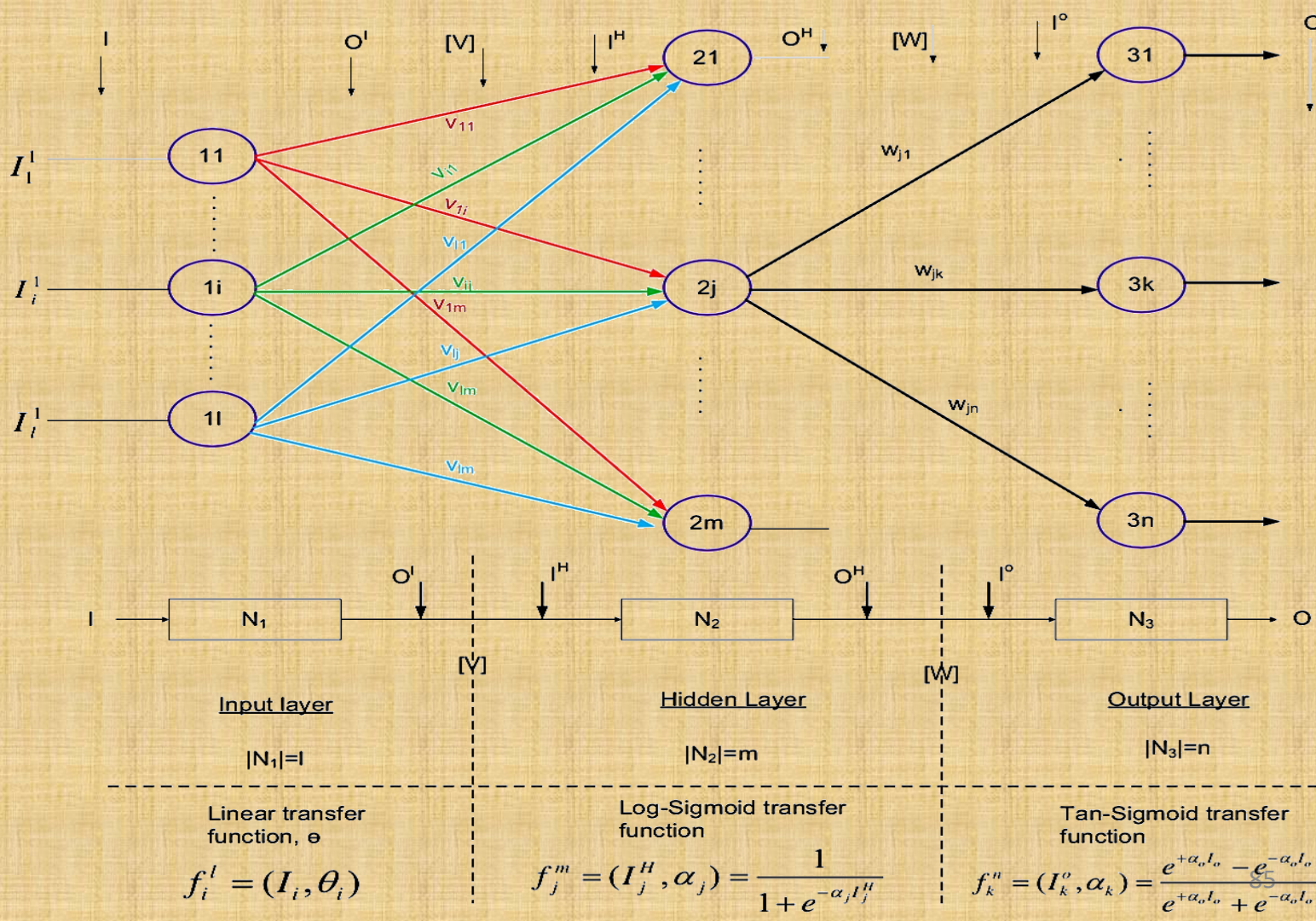
- Multilayer feed forward neural network

Training MLFFNNs

Training multilayer feed forward neural network

- Like single layer feed forward neural network, supervisory training methodology is followed to train a multilayer feed forward neural network.
- Before going to understand the training of such a neural network, we redefine some terms involved in it.
- A block diagram and its configuration for a three layer **multilayer FF NN of type $l - m - n$** is shown in the next slide.

MLFFNN



Specifying a MLFFNN

- For simplicity, we assume that all neurons in a particular layer follow the same transfer function and different layers follow their respective transfer functions as shown in the configuration.
- Let us consider a specific neuron in each layer say i -th, j -th and k -th neurons in the input, hidden and output layer, respectively.
- Also, let us denote the weight between i -th neuron ($i = 1, 2, \dots, l$) in input layer to j -th neuron ($j = 1, 2, \dots, m$) in the hidden layer is denoted by v_{ij} .

Hidden layer computation

- Let us consider any j -th neuron in the hidden layer.
- Since the output of the input layer's neurons are the input to the j -th neuron and the j -th neuron follows the **log-sigmoid transfer function**, we have

$$O_j^H = \frac{1}{1 + e^{-\alpha_H \cdot I_j^H}}$$

where $j = 1, 2, \dots, m$ and α_H is the constant co-efficient of the transfer function.

Output layer computation

Now, we estimate the output of the k -th neuron in the output layer. We consider the **tan-sigmoid transfer function**.

$$O_k = \frac{e^{\alpha_o \cdot I_k^o} - e^{-\alpha_o \cdot I_k^o}}{e^{\alpha_o \cdot I_k^o} + e^{-\alpha_o \cdot I_k^o}}$$

where $k = 1, 2, \dots, n$

Back Propagation Algorithm

- We consider the most popular algorithm called Back- Propagation algorithm, which is a supervised learning.
- The principle of the Back-Propagation algorithm is based on the error-correction with Steepest-descent method.
- We first discuss the method of steepest descent followed by its use in the training algorithm.

Method of Steepest Descent

- Supervised learning is, in fact, error-based learning.
- In other words, with reference to an external (teacher) signal (i.e. target output) it calculates error by comparing the target output and computed output.
- Based on the error signal, the neural network should modify its configuration, which includes synaptic connections, that is , the weight matrices.

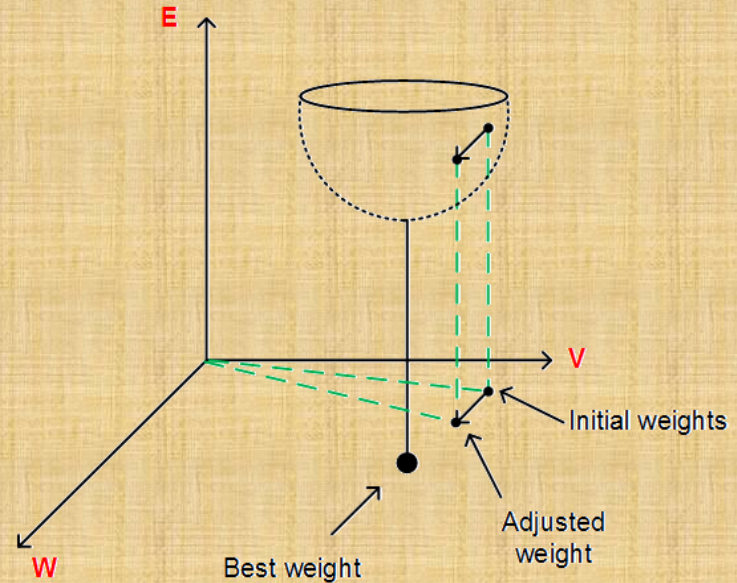
Method of Steepest Descent

- It should try to reach to a state, which yields minimum error.
- In other words, its searches for a suitable values of parameters minimizing error, given a training set.
- Note that, this problem turns out to be an optimization problem.

Method of Steepest Descent



(a) Searching for a minimum error



(b) Error surface with two parameters V and W

Method of Steepest Descent

- For simplicity, let us consider the connecting weights are the only design parameter.
- Suppose, V and W are the weights parameters to hidden and output layers, respectively.
- Thus, given a training set of size N , the error surface, E can be represented as

$$E = \sum_{i=1}^N e^i(V, W, I_i)$$

where I_i is the i -th input pattern in the training set and $e^i(\dots)$ denotes the error computation of the i -th input.

Method of Steepest Descent

- Now, we will discuss the steepest descent method of computing error, given a changes in V and W matrices.
- Suppose, A and B are two points on the error surface. The vector $\overrightarrow{AB}$ can be written as

$$\overrightarrow{AB} = (V_{i+1} - V_i) \cdot \bar{x} + (W_{i+1} - W_i) \cdot \bar{y} = \Delta V \cdot \bar{x} + \Delta W \cdot \bar{y}$$

The gradient of $\overrightarrow{AB}$ can be obtained as

$$e_{\overrightarrow{AB}} = \frac{\partial E}{\partial V} \cdot \bar{x} + \frac{\partial E}{\partial W} \cdot \bar{y}$$

Method of Steepest Descent

- Hence, the **unit vector** in the direction of gradient is

$$\bar{e}_{AB} = \frac{1}{|e_{AB}|} \left[\frac{\partial E}{\partial V} \cdot \bar{x} + \frac{\partial E}{\partial W} \cdot \bar{y} \right]$$

we can alternatively represent the distance vector AB as

$$\overrightarrow{AB} = \eta \left[\frac{\partial E}{\partial V} \cdot \bar{x} + \frac{\partial E}{\partial W} \cdot \bar{y} \right]$$

where $\eta = \frac{k}{|e_{AB}|}$ and k is a constant

Calculation of error in a neural network

- Let us consider any k -th neuron at the output layer. For an input pattern $I_i \in T_I$ (input in training) the target output T_{O_k} of the k -th neuron be T_{O_k} .
- Then, the error e_k of the k -th neuron is defined corresponding to the input I_i as

$$e_k = \frac{1}{2} (T_{O_k} - O_{O_k})^2$$

where O_{O_k} denotes the observed output of the k -th neuron.

Supervised learning : Back-propagation algorithm

- The **back-propagation algorithm** can be followed to train a neural network to set its topology, connecting weights, bias values and many other parameters.
- In this present discussion, we will **only consider updating weights**. Thus, we can write the error E corresponding to a particular training scenario T as a function of the variable V and W . That is

$$E = f(V, W, T)$$

Supervised learning : Back-propagation algorithm

- In BP algorithm, this error E is to be minimized using the gradient descent method. We know that according to the gradient descent method, the changes in weight value can be given as

$$\Delta V = -\eta \frac{\partial E}{\partial V} \quad (1)$$

$$\Delta W = -\eta \frac{\partial E}{\partial W} \quad (2)$$

Supervised learning : Back-propagation algorithm

- Note that — *ve* sign is used to signify the fact that if $\frac{\partial E}{\partial V}$ (or $\frac{\partial E}{\partial W}$) > 0 , then we have to decrease V and vice-versa.
- Let v_{ij} (and w_{jk}) denotes the weights connecting i -th neuron (at the input layer) to j -th neuron (at the hidden layer) and connecting j -th neuron (at the hidden layer) to k -th neuron (at the output layer).
- Also, let e_k denotes the error at the k -th neuron with observed output as $O_{O_k}^o$ and target output $T_{O_k}^o$ as per a sample input $I \in TI$.

Calculation of $\bar{w}_{jk}$

We can calculate $\frac{\partial e_k}{\partial w_{jk}}$ using the **chain rule of differentiation** as stated below.

$$\frac{\partial e_k}{\partial w_{jk}} = \frac{\partial e_k}{\partial O_{o_k^o}} \cdot \frac{\partial O_{o_k^o}}{\partial I_k^o} \cdot \frac{\partial I_k^o}{\partial w_{jk}} \quad (5)$$

Now we have

$$e_k = \frac{1}{2} \left(T_{o_k^o} - O_{o_k^o} \right)^2 \quad (6)$$